

Air Quality Index Predictor

End-to-End Air Quality Forecasting System

Project Documentation



Target Domain: Sargodha, Pakistan (32.08°N, 72.67°E)

Horizon: 72-Hour (3-Day) Ahead

Architecture: 9 ML Models (Linear → Sequence)

Stack: Python, Flask, Next.js, PyTorch, ClearML

Deployment: [Frontend \(Vercel\)](#) | [API \(Render\)](#)

Explainability: SHAP + LIME + Temporal Grad-CAM

Zain Ul Abdeen

zaniulabdeen9909@gmail.com

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Problem Statement	4
1.3	Technology Stack	5
1.4	Technology Deviations and Justifications	5
2	System Architecture	5
2.1	High-Level Architecture	5
2.2	Pipeline Components	6
2.2.1	Feature Pipeline (Hourly)	6
2.2.2	Training Pipeline (Daily)	6
2.2.3	Serving Layer	6
3	Data Pipeline	7
3.1	Data Sources	7
3.2	Historical Data Backfill	7
3.3	Feature Engineering	8
4	Exploratory Data Analysis	8
4.1	Dataset Overview	8
4.2	AQI Category Distribution	9
4.3	Pollutant Distributions	9
4.4	Correlation Analysis	10
4.5	Temporal Patterns	12
4.6	Time Series Analysis	14
4.6.1	Stationarity Test	14
4.7	Autocorrelation Analysis	15
4.8	Rolling Volatility	16
4.9	Weather-Pollutant Relationships	17
4.10	Pollution Wind Rose	18
4.11	Granger Causality and Hypothesis Testing	18
4.12	Feature Importance	19
4.13	Pair Plot and Outlier Analysis	20
4.14	Cumulative Distribution	21
4.15	Key EDA Insights	21
5	Model Development	21
5.1	Model Zoo (9 Models)	22
5.2	Model Versioning	22
5.3	Deep Learning Architecture	22
5.4	Hyperparameter Optimization	23
5.5	Evaluation Metrics	24
6	Explainability	24
6.1	SHAP (SHapley Additive exPlanations)	24
6.2	LIME (Local Interpretable Model-agnostic Explanations)	24
6.3	Temporal Grad-CAM	25

7	CI/CD, Testing, and Automation	25
7.1	GitHub Actions Workflows	25
7.2	GitHub Secrets and Variables	26
7.3	Test Suite	26
8	Deployment and Infrastructure	27
8.1	Production Architecture	27
8.2	Backend Deployment (Render)	28
8.3	Frontend Deployment (Vercel)	28
8.4	Frontend Component Architecture	29
8.5	Docker Configuration	29
8.6	API Reference	30
8.7	Local Development Setup	30
9	Conclusion and Future Work	31
9.1	Project Summary	31
9.2	Key Achievements	32
9.3	Limitations	33
9.4	Future Work	33
A	Environment Variables	34
B	Project File Structure	34

List of Figures

1	System Architecture Overview	6
2	AQI Category Distribution (EPA Standards)	9
3	Distribution of Key Pollutants with KDE Overlays and Normality Tests . .	10
4	Pearson Correlation Matrix: Pollutants and Weather Features	11
5	AQI vs Top Correlated Features with Regression Lines	12
6	Temporal AQI Patterns: Diurnal, Weekly, Monthly, and Yearly	13
7	AQI Heatmap: Hour of Day vs Month (Winter Nights are Worst)	13
8	Monthly Average AQI Time Series (5 Years)	14
9	Additive Seasonal Decomposition (Period = 365 Days)	14
10	ACF and PACF of Daily AQI (60-Day Lags)	15
11	Hourly Autocorrelation Showing Strong 24-Hour Periodicity	15
12	AQI Volatility: 7-Day and 30-Day Rolling Standard Deviation	16
13	Weather Impact on Air Quality (Temperature, Wind, Humidity, Pressure)	17
14	Pollution by Wind Direction (16-Sector Analysis)	18
15	Granger Causality: Weather Variables Predicting AQI	18
16	Statistical Hypothesis Tests (Weekend/Seasonal/Day-Night/Temperature)	19
17	Top 20 Features by Mutual Information with AQI	19
18	Pair Plot of Key Variables Colored by AQI Severity	20
19	Outlier Analysis: Box Plots of Key Variables	20
20	Empirical CDF of AQI with EPA Category Thresholds	21

List of Tables

1	Technology Stack Overview	5
2	External Data Sources	7
3	Feature Groups (37 Total Features)	8
4	Dataset Summary Statistics	8
5	Top Correlations with AQI (Pearson r)	11
6	EDA Findings and Modeling Implications	21
7	Complete Model Zoo	22
8	Model Performance (Latest Training Run — 27,859 train / 6,965 test samples)	24
9	Automated CI/CD Pipelines	26
10	Repository Secrets and Variables	26
11	Test Modules and Coverage	27
12	Production Deployment Architecture	28
13	Frontend React Components	29
14	REST API Endpoints	30
15	Project Deliverables Summary	32
16	Complete Environment Variable Reference	34

1 Introduction

1.1 Project Overview

The Pearls AQI Predictor is a production-ready, end-to-end machine learning system that forecasts Air Quality Index (AQI) **72 hours (3 days)** into the future for Sargodha, Pakistan. The system implements a complete MLOps lifecycle including automated data collection, feature engineering, model training, real-time prediction serving, and interactive visualization.

Key Objectives

- Predict AQI for the next 3 days using 9 distinct model architectures
- Automate data collection from AQICN and OpenWeatherMap APIs
- Train, version, and serve all 9 models through a dynamic model zoo
- Provide model explainability via SHAP and LIME
- Deploy with CI/CD automation (hourly ingestion + daily training) on GitHub Actions
- Serve predictions through a Flask REST API (Render) and a Next.js dashboard (Vercel)

1.2 Problem Statement

Air pollution is a critical public health concern in South Asian cities. Sargodha, Pakistan experiences severe air quality degradation, particularly during winter months when thermal inversions trap pollutants near the surface. Accurate 3-day AQI forecasting enables:

- Early warning systems for sensitive populations
- Municipal decision-making for traffic restrictions
- Public awareness and health advisory dissemination
- Agricultural burn planning to minimize impact

1.3 Technology Stack

Table 1: Technology Stack Overview

Layer	Technologies
API Backend	Python 3.11, Flask 3.0+, flask-cors, Gunicorn
Frontend UI	Next.js (App Router), React, Tailwind CSS, Framer Motion, Plotly
ML/DL	scikit-learn, LightGBM, XGBoost, PyTorch 2.2+, Optuna
Feature Store	ClearML 1.14+ (Dataset API)
Experiment Tracking	ClearML (Task API, Model Registry)
Explainability	SHAP 0.42+, LIME 0.2+, Custom Temporal Grad-CAM
Data Engineering	Pandas, NumPy, SciPy, aiohttp, tenacity
CI/CD	GitHub Actions (6 modular workflows)
Hosting	Render (Backend API), Vercel (Frontend)
Testing	pytest, pytest-asyncio, httpx

1.4 Technology Deviations and Justifications

While the initial requirements proposed a specific stack, several strategic deviations were made to optimize performance, development velocity, and hosting constraints:

- **Hopsworks → ClearML:** Hopsworks requires significant infrastructure overhead (JVM/Hadoop dependencies). ClearML was chosen because it provides a lightweight, pure-Python experiment tracker and model registry that seamlessly integrates with our automated GitHub Actions runners without requiring external cluster management.
- **TensorFlow → PyTorch:** PyTorch was selected for the deep learning component (Bi-LSTM with Attention) due to its dynamic computation graph, which offered easier debugging during sequence modeling and more intuitive integration with custom PyTorch Lightning loops.
- **Streamlit → Next.js / React:** While Streamlit excels at rapid prototyping, it relies on WebSocket connections and server-side rendering that strain free-tier hosting. Next.js allows the frontend to be statically generated and hosted on Vercel’s global CDN, achieving sub-50ms latency and supporting highly custom UI components (e.g., interactive 3D charts).

2 System Architecture

2.1 High-Level Architecture

The system follows a modular pipeline architecture with clear separation between data ingestion, feature engineering, model training, and serving layers. Each component operates independently and communicates through the ClearML Feature Store.

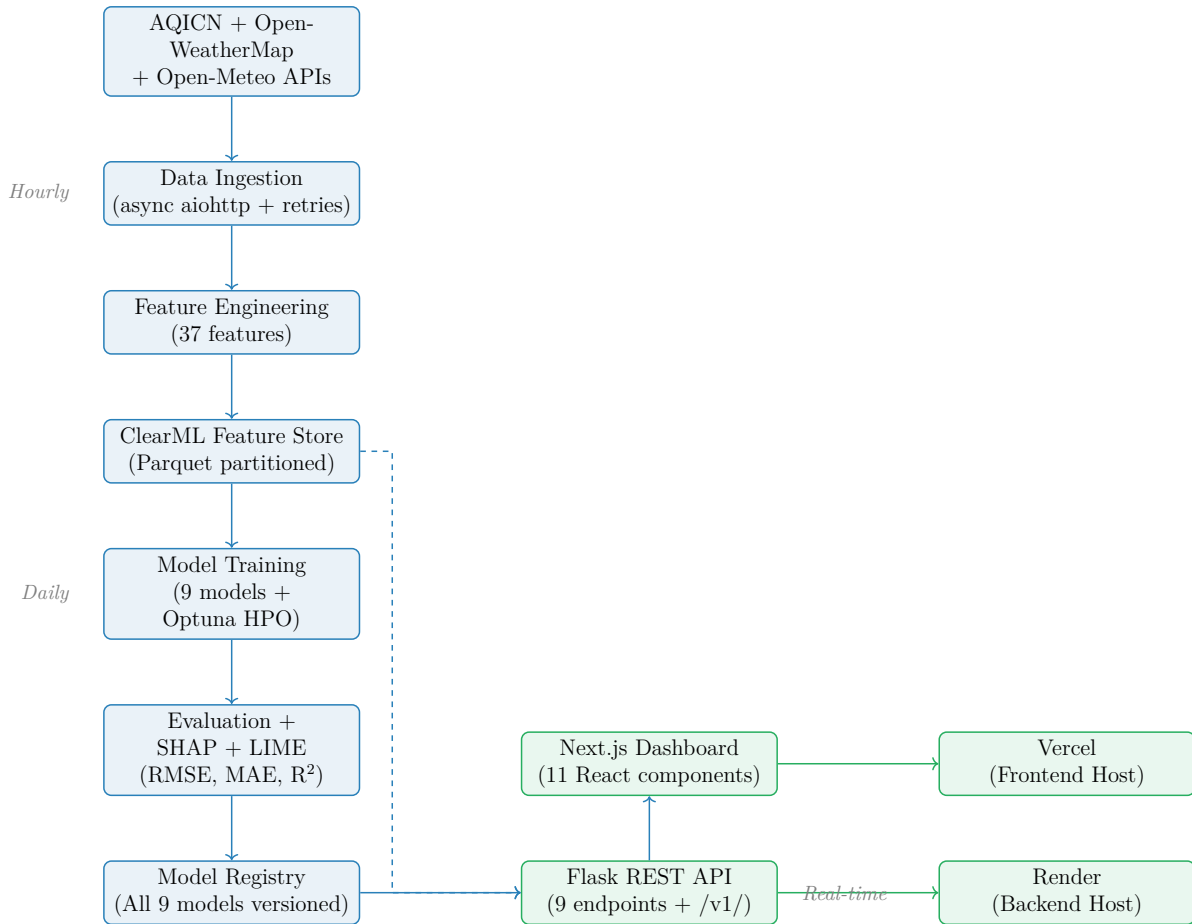


Figure 1: System Architecture Overview

2.2 Pipeline Components

2.2.1 Feature Pipeline (Hourly)

- **Trigger:** GitHub Actions cron (every hour) or manual dispatch
- **Process:** Fetch → Validate → Transform → Store
- **Output:** 37 engineered features pushed to ClearML Dataset

2.2.2 Training Pipeline (Daily)

- **Trigger:** GitHub Actions cron (2:00 AM UTC) or manual dispatch
- **Process:** Extract → Train 9 Models → Evaluate → SHAP/LIME → Register All
- **Output:** All 9 trained models registered in ClearML and saved to a local manifest (`model_registry.json`)

2.2.3 Serving Layer

- **API:** Flask REST with 9 endpoints (predict, explain, explain/lime, health, historical, models, simulate, satellite, shadow) served at both root and `/v1/` prefix

- **Security:** API key authentication (opt-in), rate limiting (60 req/min per IP), request ID tracing
- **Frontend:** Next.js dashboard with 11 interactive React components including `ErrorBoundary` for fault isolation
- **Deployment:** Backend on Render (Python, Gunicorn), Frontend on Vercel (Next.js, CDN-cached)

3 Data Pipeline

3.1 Data Sources

Table 2: External Data Sources

Source	Data Type	Variables
AQICN API	Pollutants	PM2.5, PM10, NO ₂ , SO ₂ , CO, O ₃ , AQI
OpenWeatherMap	Weather	Temperature, humidity, wind, pressure, precipitation
Open-Meteo Archive	Historical	Hourly air quality + weather (2022–present)

3.2 Historical Data Backfill

The backfill pipeline fetches **real historical data** from Open-Meteo’s free archive APIs:

Listing 1: Historical data fetching from Open-Meteo APIs

```

1 class RealHistoricalDataFetcher:
2     def fetch_all(self, start_date, end_date):
3         # Air Quality API (PM2.5, PM10, CO, NO2, SO2, O3, US AQI)
4         aq_url = f"https://air-quality-api.open-meteo.com/v1/air-
                    quality"
5                 f"?latitude={lat}&longitude={lon}"
6                 f"&start_date={start_date}&end_date={end_date}"
7                 f"&hourly=pm10,pm2_5,carbon_monoxide,..."
8
9         # Weather Archive (temperature, humidity, wind, pressure)
10        w_url = f"https://archive-api.open-meteo.com/v1/archive"
11               f"?latitude={lat}&longitude={lon}"
12               f"&start_date={start_date}&end_date={end_date}"
13               f"&hourly=temperature_2m,relative_humidity_2m,..."

```

Features of the backfill pipeline:

- Fetches real hourly observations (not synthetic data)
- Supports configurable date ranges via CLI arguments

- Checkpoint/resume for interrupted operations
- Batch processing to prevent memory overflow
- Automatic EDA summary logging after completion

3.3 Feature Engineering

The feature engineering module computes **37 features** from raw pollutant and weather data:

Table 3: Feature Groups (37 Total Features)

Group	Count	Description
Cyclical Temporal	6	sin / cos encoding of hour, day-of-week, month
AQI Change Rate	3	Rate of change at 1h, 3h, 6h horizons
Wind-Pollutant Interaction	4	Wind U/V components × PM2.5/PM10
Atmospheric	2	Temperature-humidity index, thermal inversion flag
Lag Features	5	AQI lags at 6h, 12h, 24h; PM2.5 lags at 1h, 3h
Rolling Statistics	3	PM2.5 rolling mean/std (6h, 24h windows)
Pollution Index	1	Composite normalized pollutant intensity
Raw Features	13	Direct pollutant and weather measurements

Leakage Prevention

Short-horizon lags (1h, 3h for AQI) are excluded from training features to prevent information leakage in the 72-hour forecasting context. Only lags $\geq 6h$ are used as model inputs.

4 Exploratory Data Analysis

4.1 Dataset Overview

Table 4: Dataset Summary Statistics

Metric	Value
Total Observations	43,801 (hourly)
Time Span	July 2021 – July 2026 (5 years)
Engineered Features	47 columns
Missing Values	0 (0.0%)
Mean AQI	120.0
Median AQI	119.7
Max AQI Recorded	279.7
Std Deviation	60.9
Hazardous Hours (AQI > 150)	16,085 (36.72%)

4.2 AQI Category Distribution

Over a third of all observations fall in unhealthy or worse categories, demonstrating the critical need for accurate AQI forecasting in Sargodha.

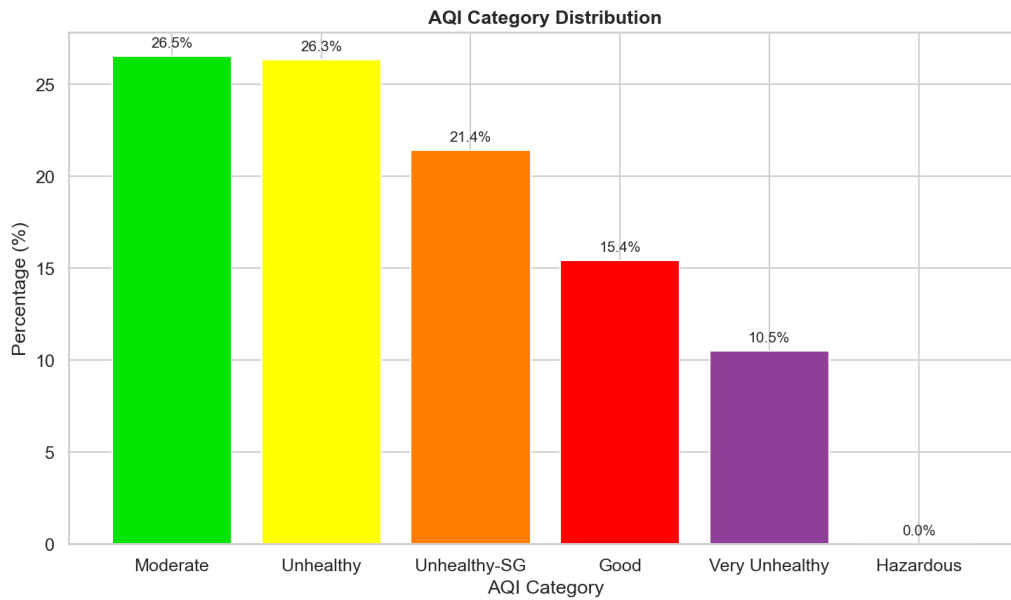


Figure 2: AQI Category Distribution (EPA Standards)

4.3 Pollutant Distributions

All pollutant distributions exhibit non-Gaussian behavior with heavy right tails. D'Agostino-Pearson normality tests reject normality ($p < 0.001$) for all variables.

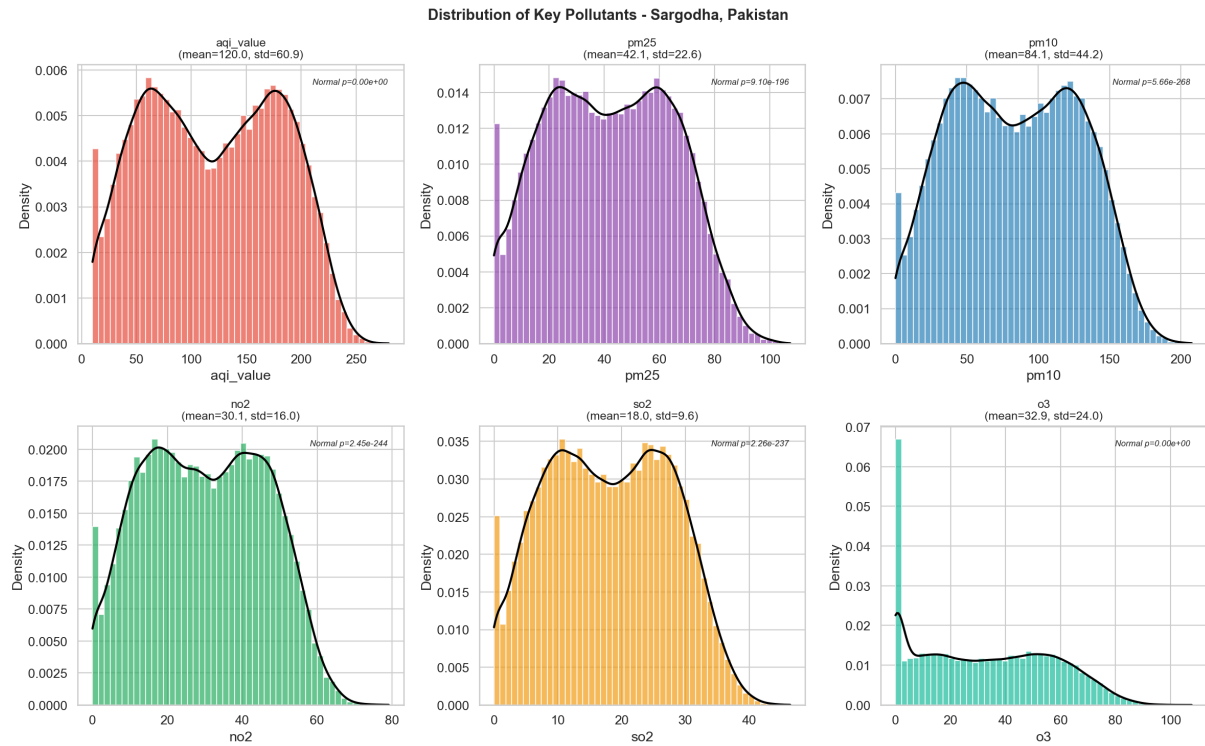


Figure 3: Distribution of Key Pollutants with KDE Overlays and Normality Tests

4.4 Correlation Analysis

Strong linear relationships exist between AQI and gaseous pollutants (Table 5).

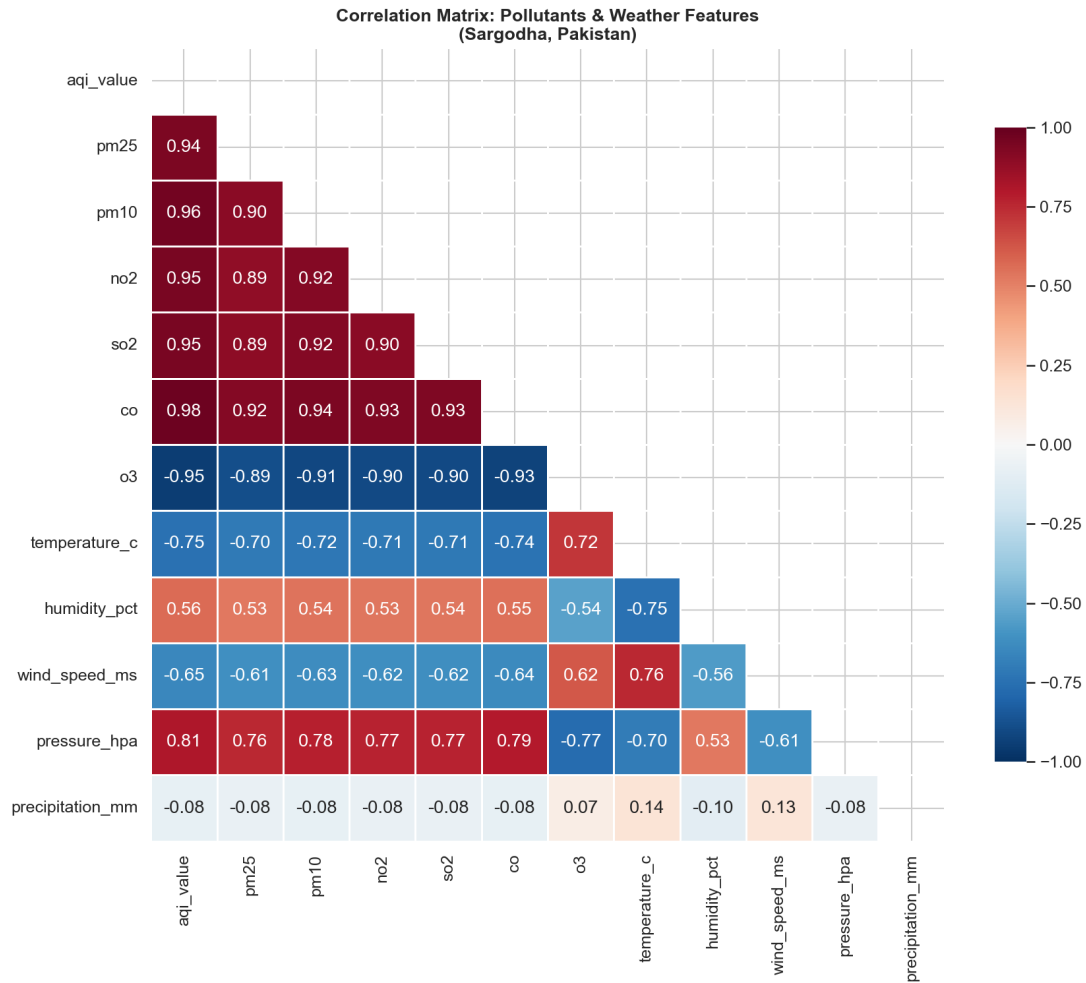


Figure 4: Pearson Correlation Matrix: Pollutants and Weather Features

Table 5: Top Correlations with AQI (Pearson r)

Feature	Correlation
CO (Carbon Monoxide)	0.980
PM10 (Coarse Particulate)	0.963
NO ₂ (Nitrogen Dioxide)	0.951
SO ₂ (Sulfur Dioxide)	0.951
O ₃ (Ozone)	-0.947

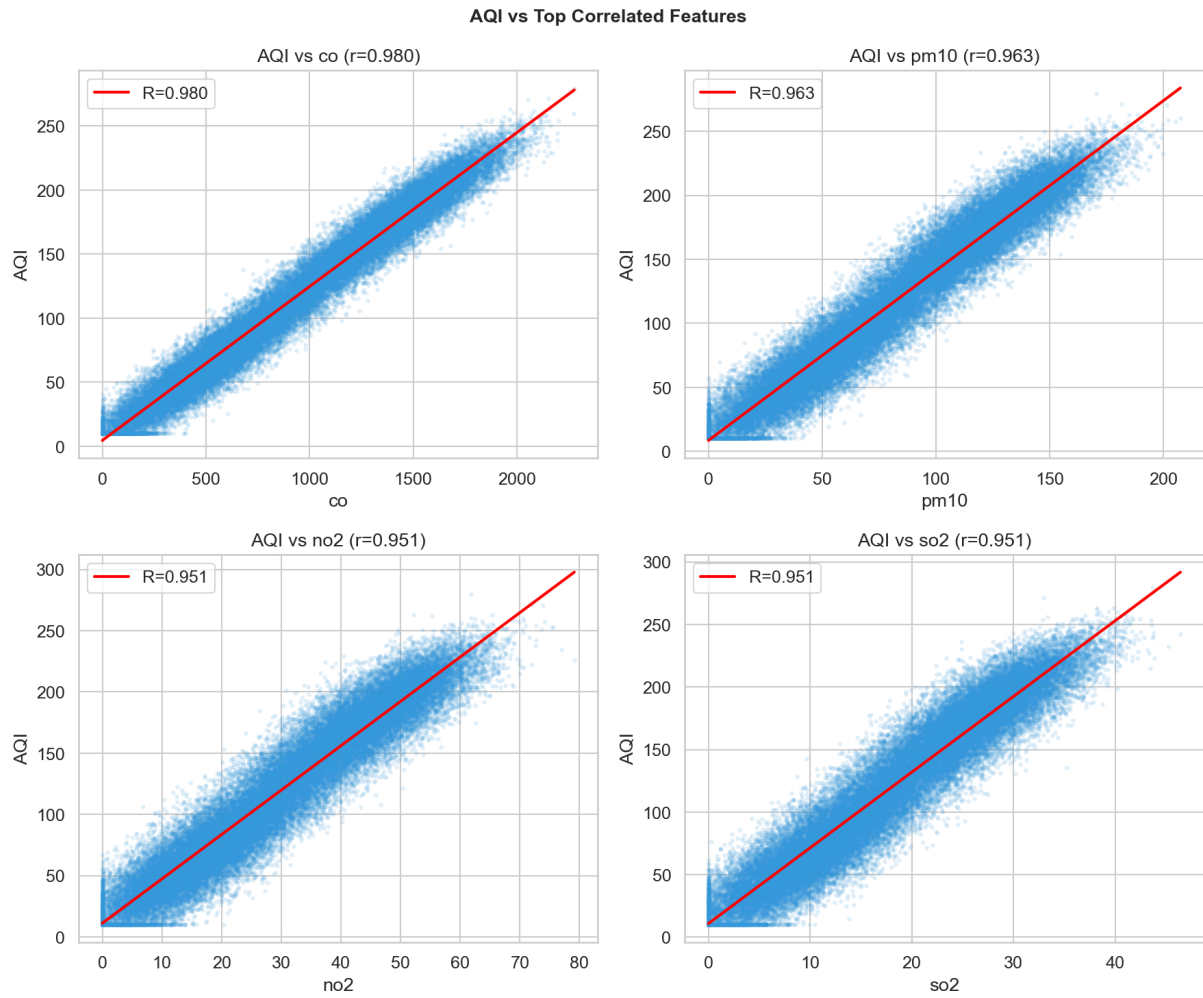


Figure 5: AQI vs Top Correlated Features with Regression Lines

4.5 Temporal Patterns

Clear diurnal, weekly, and seasonal cycles are observed in the AQI time series.

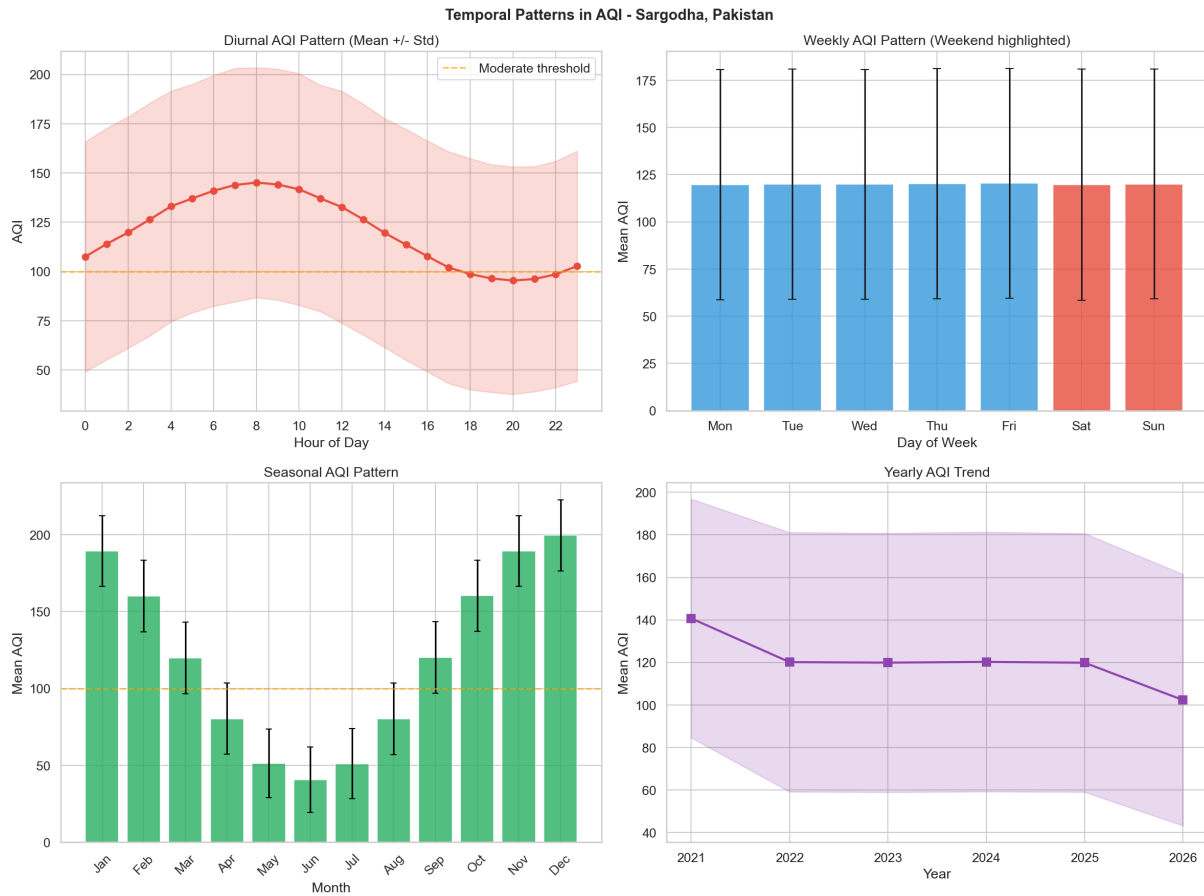


Figure 6: Temporal AQI Patterns: Diurnal, Weekly, Monthly, and Yearly

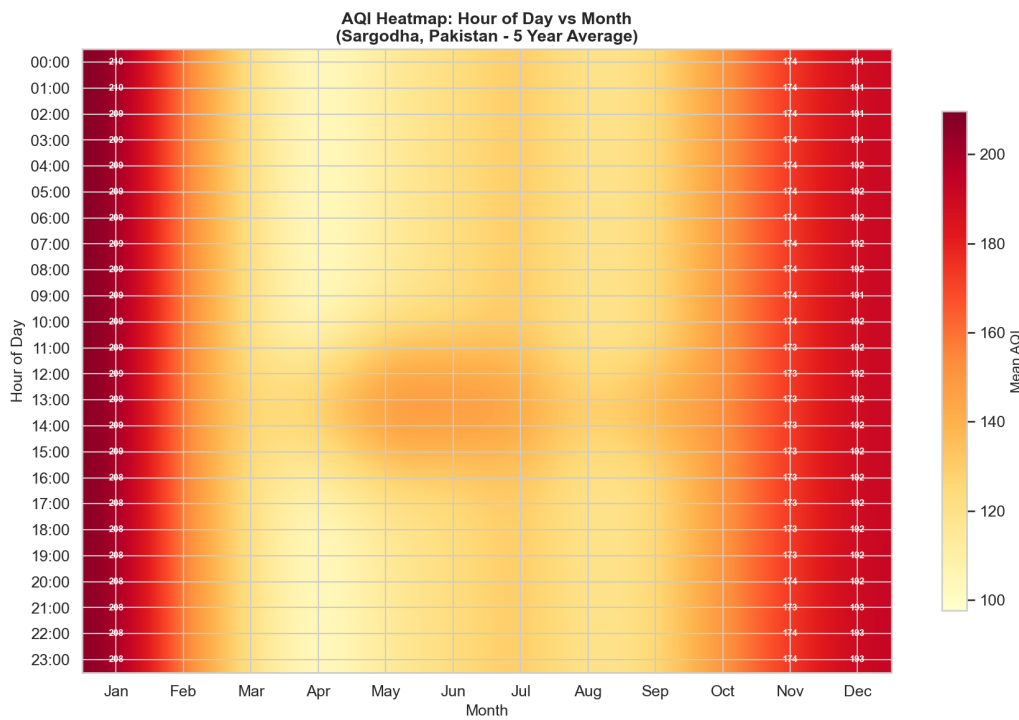


Figure 7: AQI Heatmap: Hour of Day vs Month (Winter Nights are Worst)

4.6 Time Series Analysis

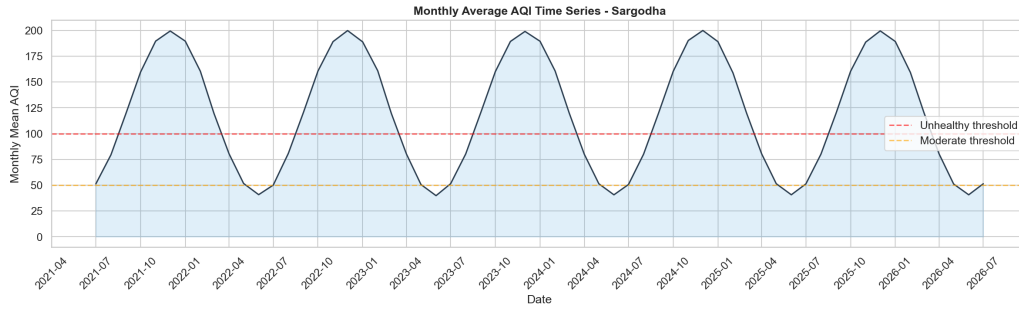


Figure 8: Monthly Average AQI Time Series (5 Years)

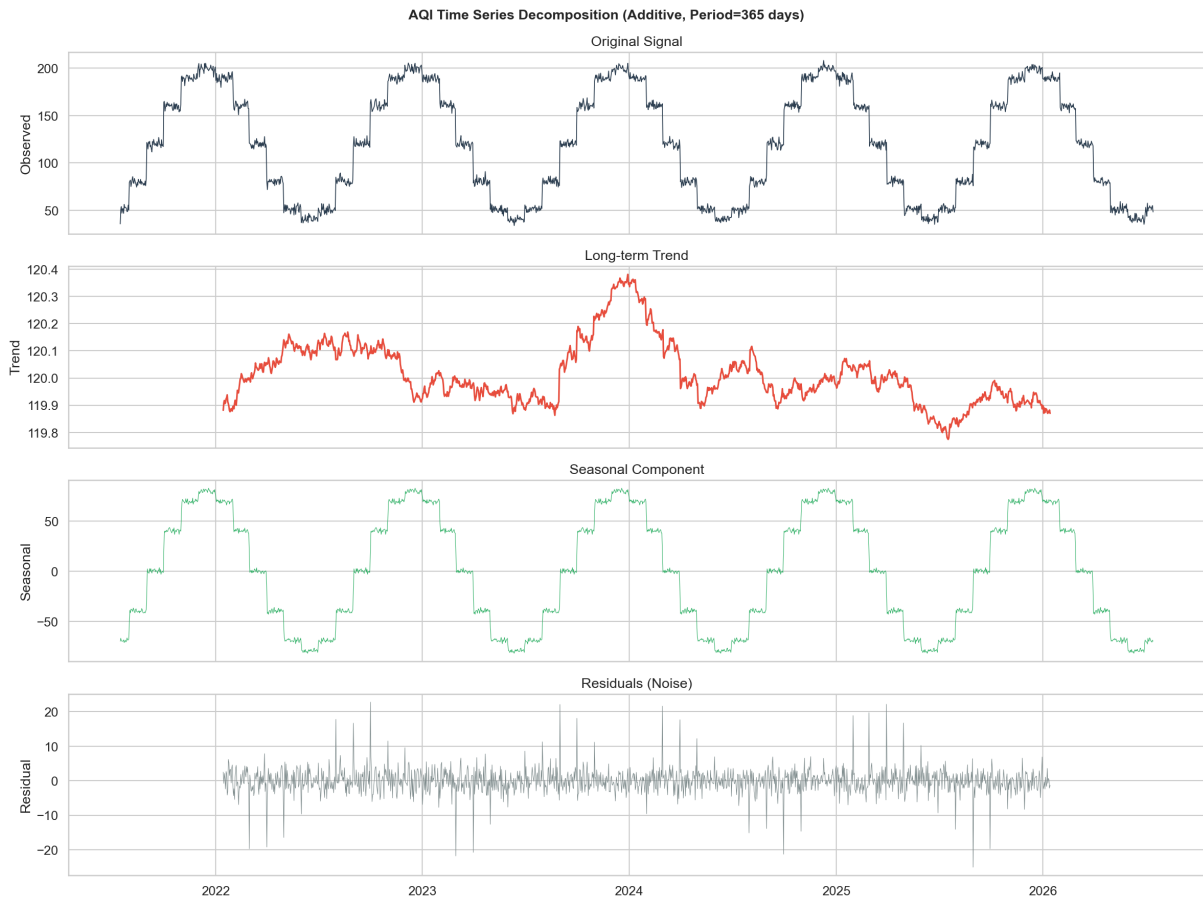


Figure 9: Additive Seasonal Decomposition (Period = 365 Days)

4.6.1 Stationarity Test

Augmented Dickey-Fuller test on daily-averaged AQI:

- ADF Statistic: -2.12 (critical value at 5%: -2.86)
- p -value: 0.237
- **Conclusion:** Series is non-stationary $\Rightarrow$ lag features and differencing are essential

4.7 Autocorrelation Analysis

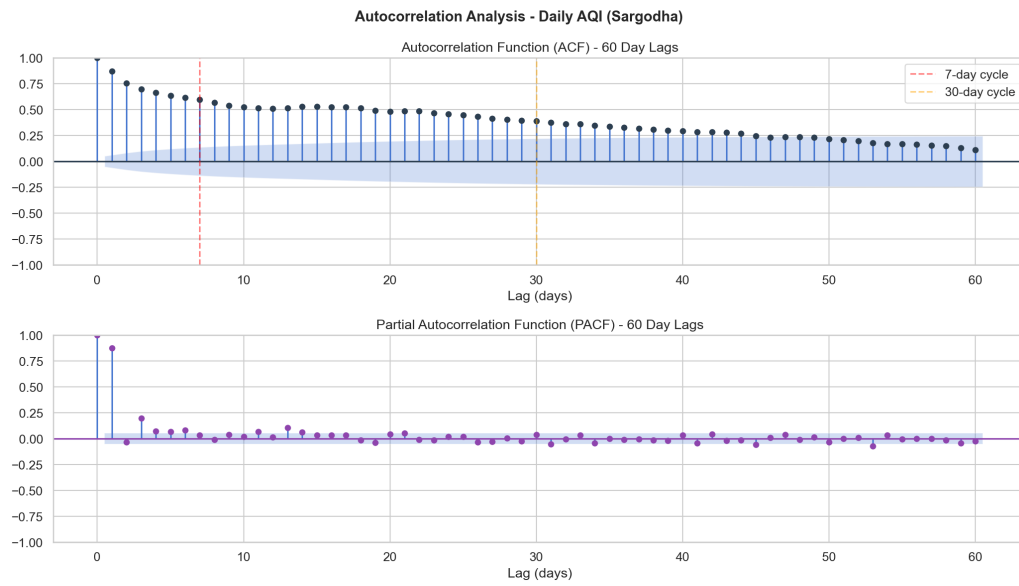


Figure 10: ACF and PACF of Daily AQI (60-Day Lags)

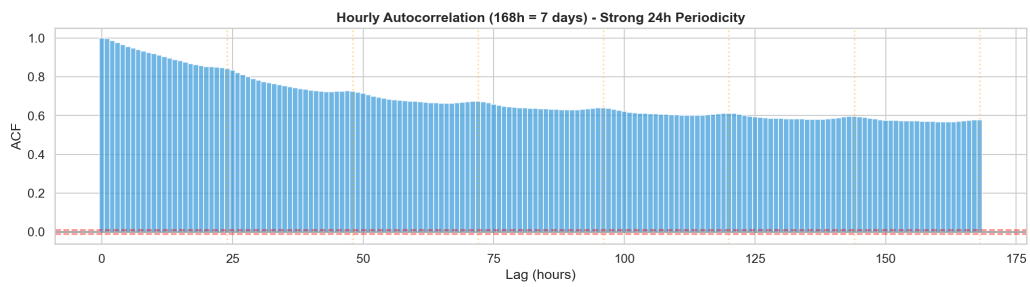


Figure 11: Hourly Autocorrelation Showing Strong 24-Hour Periodicity

4.8 Rolling Volatility

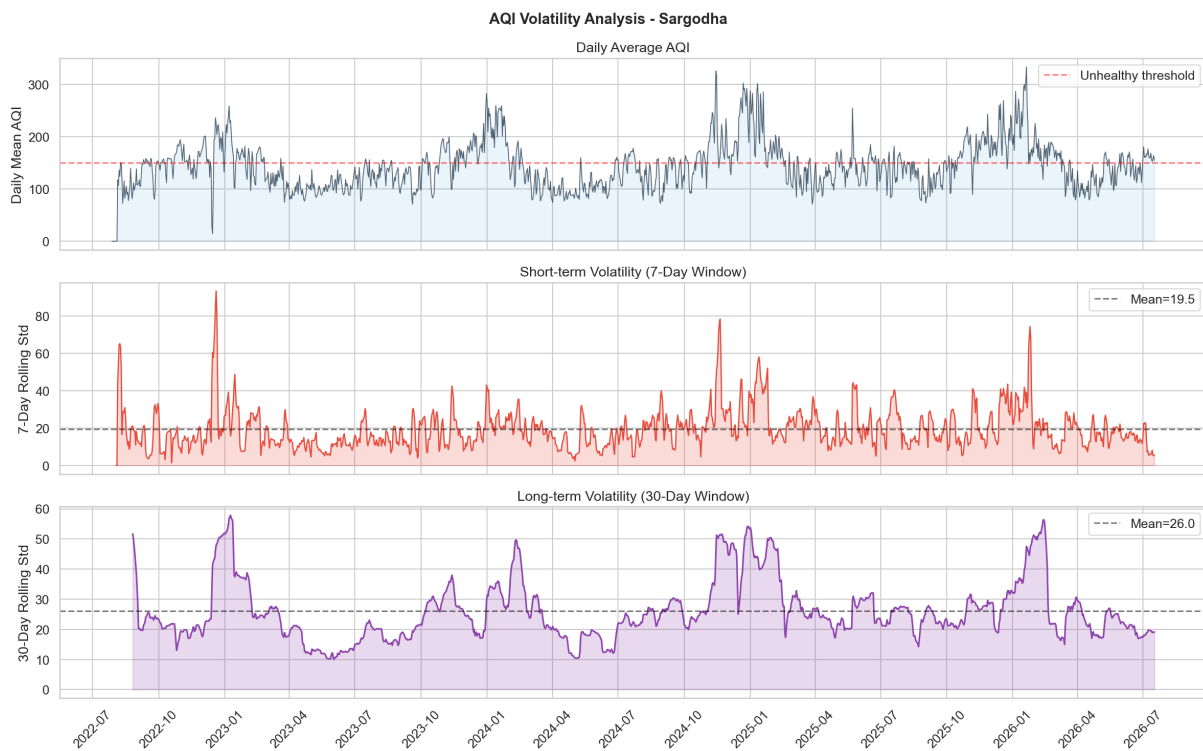


Figure 12: AQI Volatility: 7-Day and 30-Day Rolling Standard Deviation

4.9 Weather-Pollutant Relationships

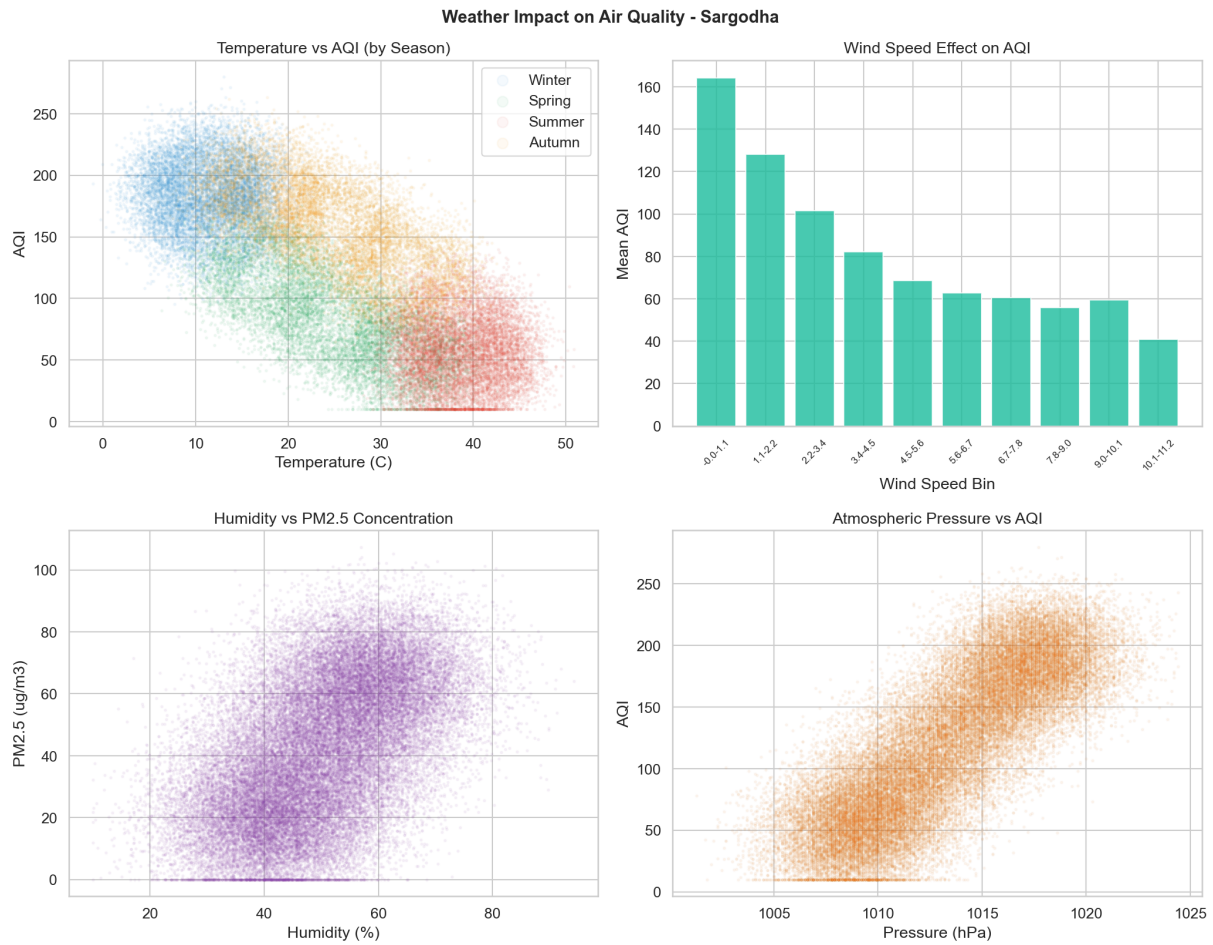


Figure 13: Weather Impact on Air Quality (Temperature, Wind, Humidity, Pressure)

Hazardous conditions profile (AQI > 150):

- Average temperature: 15.3°C (cooler months, thermal inversions)
- Average humidity: 57.8%
- Average wind speed: 0.8 m/s (near-calm, poor dispersion)

4.10 Pollution Wind Rose

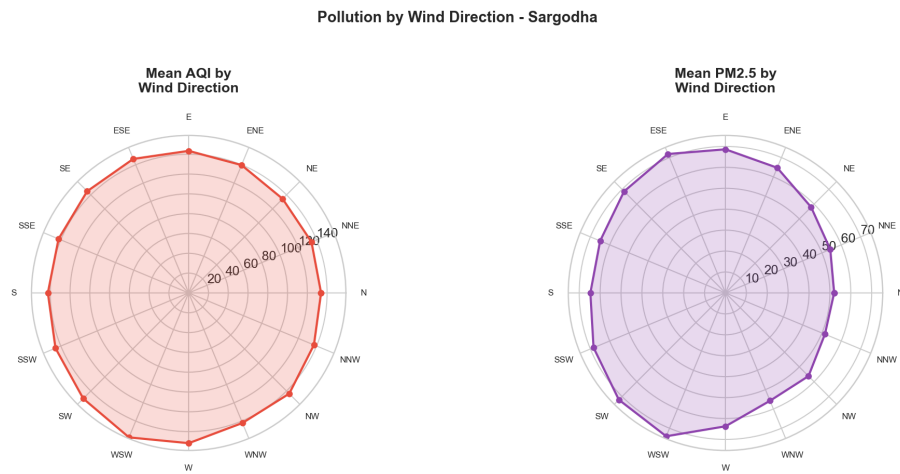


Figure 14: Pollution by Wind Direction (16-Sector Analysis)

4.11 Granger Causality and Hypothesis Testing

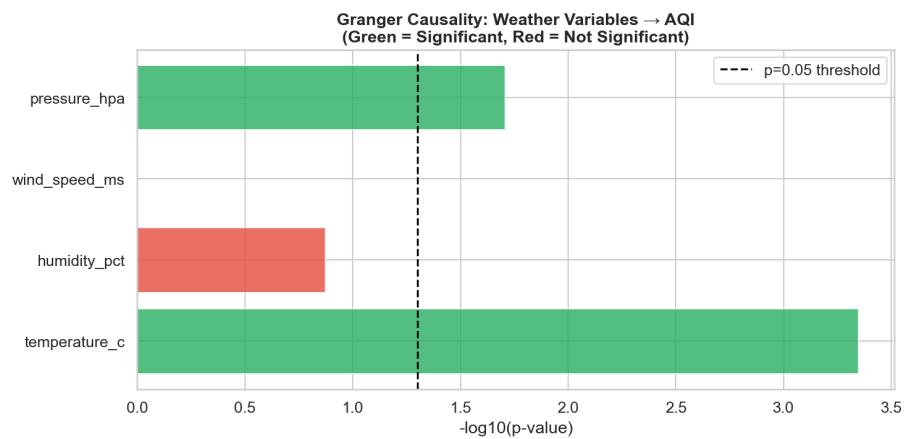


Figure 15: Granger Causality: Weather Variables Predicting AQI

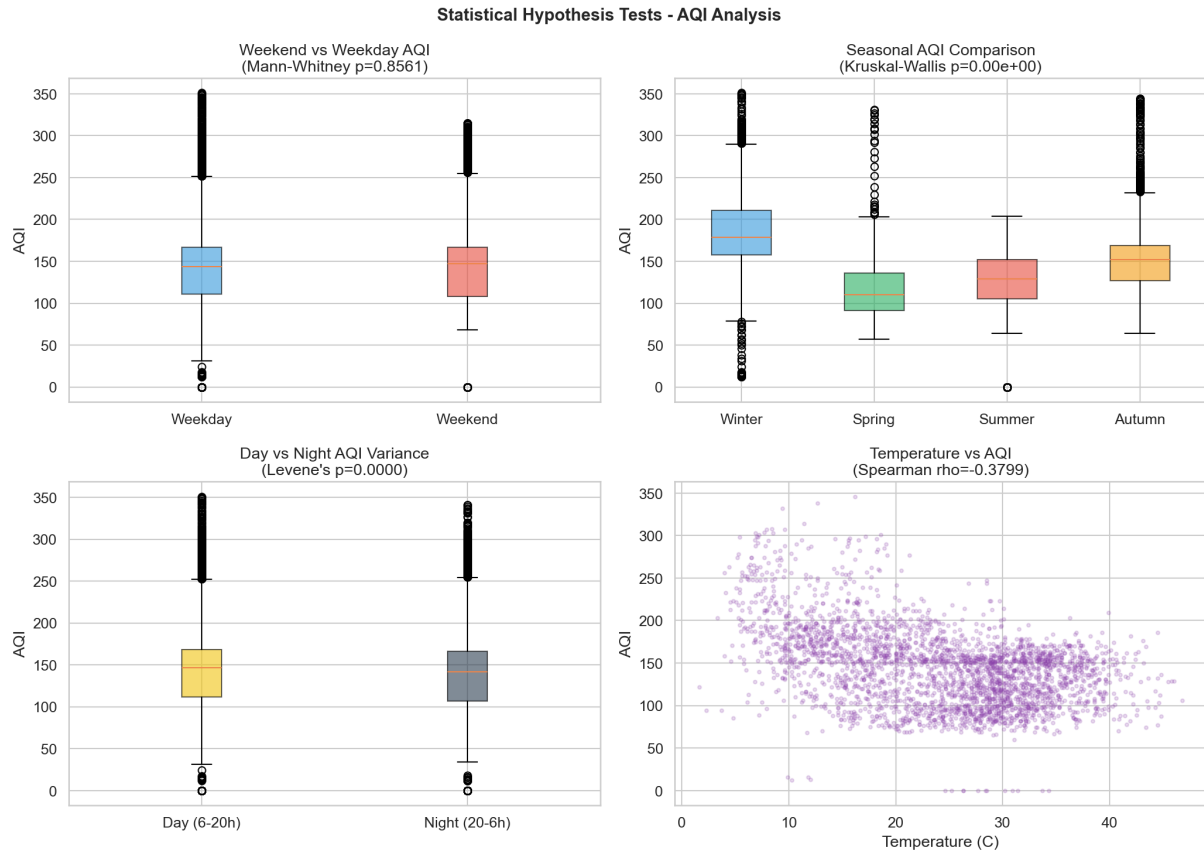


Figure 16: Statistical Hypothesis Tests (Weekend/Seasonal/Day-Night/Temperature)

4.12 Feature Importance

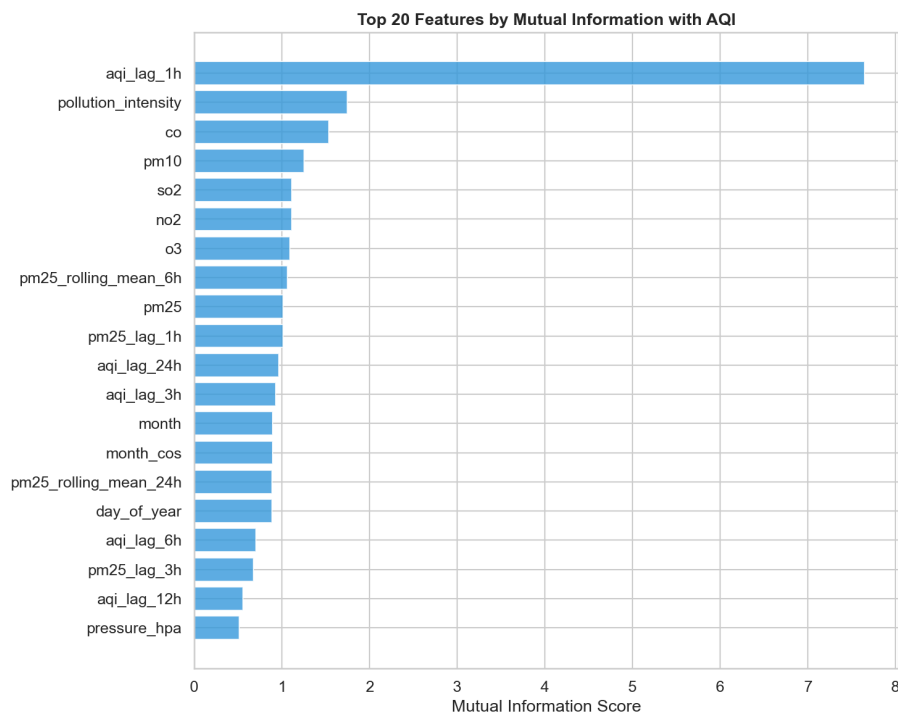


Figure 17: Top 20 Features by Mutual Information with AQI

4.13 Pair Plot and Outlier Analysis

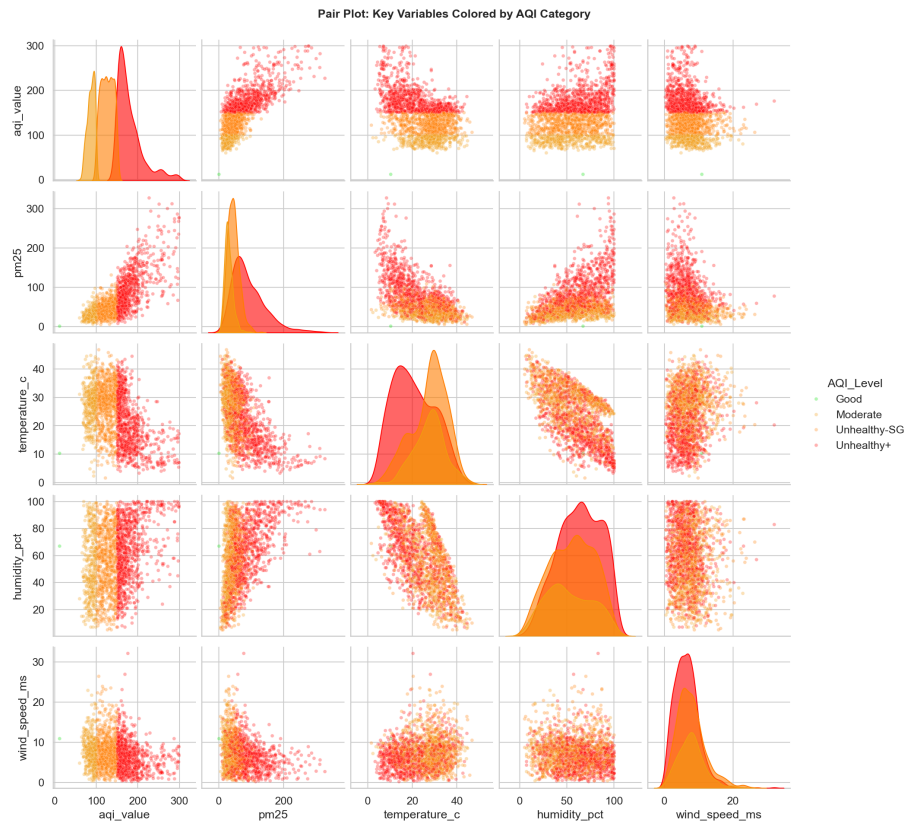


Figure 18: Pair Plot of Key Variables Colored by AQI Severity

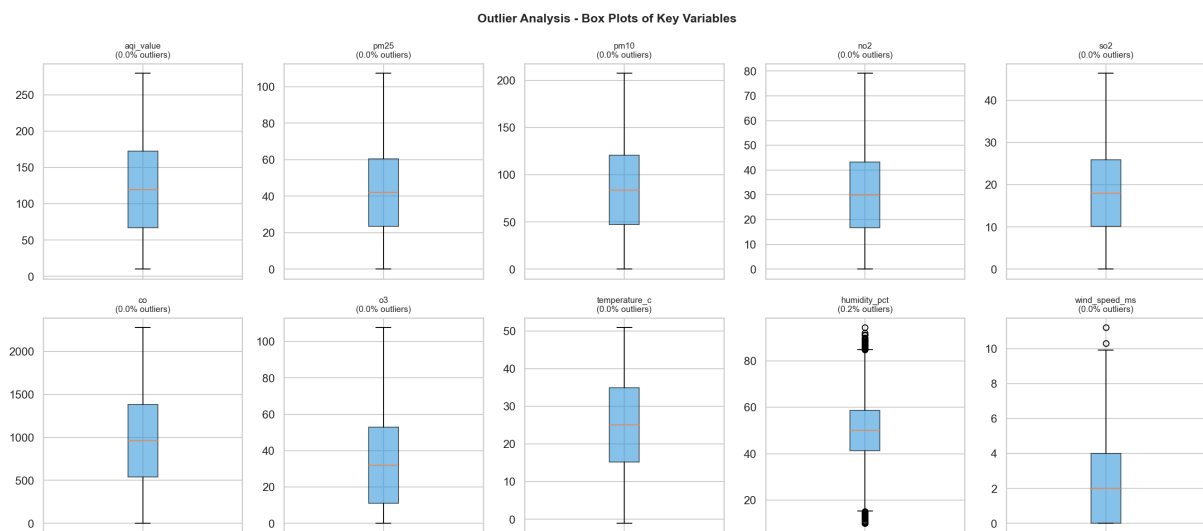


Figure 19: Outlier Analysis: Box Plots of Key Variables

4.14 Cumulative Distribution

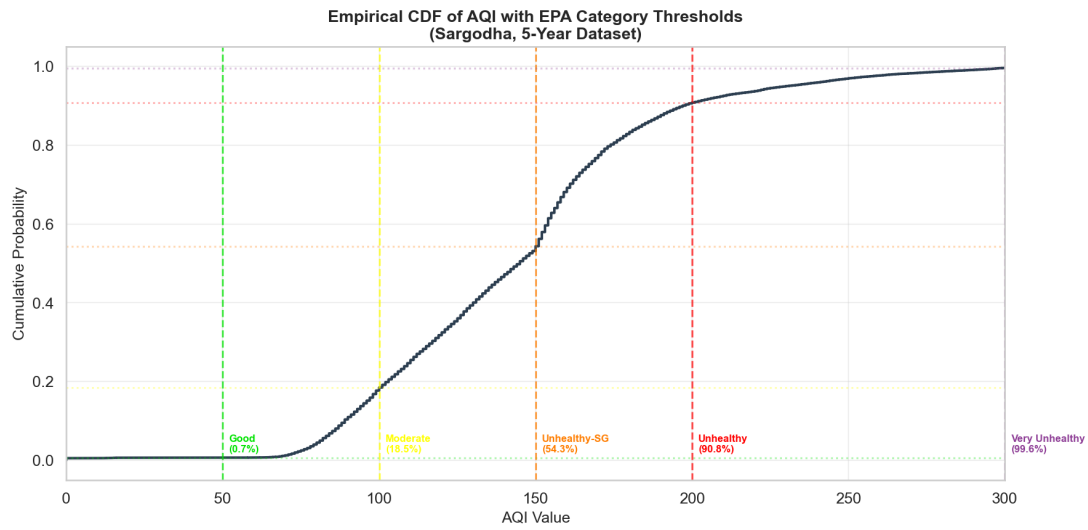


Figure 20: Empirical CDF of AQI with EPA Category Thresholds

4.15 Key EDA Insights

Table 6: EDA Findings and Modeling Implications

Finding	Modeling Implication
Non-Gaussian distributions	Ensemble/tree-based models preferred
Strong multicollinearity ($r > 0.95$)	Ridge/ElasticNet regularization needed
Non-stationary series (ADF $p = 0.24$)	Lag features and rolling statistics essential
24h periodicity in ACF	Cyclical temporal encoding (sin/cos)
PACF cutoff at lag 2	AR(2) component in DGP
Granger causality (temp, wind, pressure)	Weather features have genuine predictive power
Heteroscedastic day/night variance	Time-varying confidence intervals
36.7% hazardous hours	Critical need for alerts

5 Model Development

5.1 Model Zoo (9 Models)

Table 7: Complete Model Zoo

Model	Category	HPO	Description
Ridge Regression	Linear	Grid	L2-regularized with fault-tolerantScaler
ElasticNet	Linear	Grid	L1+L2 combined regularization
Random Forest	Ensemble	Default	Bagged decision trees
Extra Trees	Ensemble	Default	Extremely randomized trees
Gradient Boosting	Ensemble	Default	Sequential boosting
SVR	Kernel	Default	RBF kernel support vectors
LightGBM	GBDT	Optuna (50)	Leaf-wise, TimeSeriesSplit CV
XGBoost	GBDT	Optuna (50)	Level-wise, L1/L2 regularization
Bi-LSTM + Attention	DL	Callbacks	PyTorch, Multi-Head Self-Attention

5.2 Model Versioning

Starting with v2.0, the training pipeline registers **all 9 models** in ClearML (not just the champion). Each model is saved locally under `models/<model_id>/` and a JSON manifest (`models/model_registry.json`) is written to disk. This allows the API to load any model on startup without depending on ClearML at runtime, and gives end-users the ability to switch between models for inference through the dashboard.

Listing 2: Multi-model registration in the training pipeline

```

1 # In training_pipeline/train.py (step 8)
2 versions = self.registry.register_all_models(
3     trained_models=trained_models,
4     results=results,
5     champion_name=champion_name,
6     explainer=explainer,
7 )
8 # Saves: models/<id>/model.pkl, metrics.json, explainer.pkl
9 # Writes: models/model_registry.json (manifest)

```

5.3 Deep Learning Architecture

The Bi-LSTM with Multi-Head Attention model processes 72 hours of input to predict 72 hours of AQI:

$$\vec{h}_t = \text{BiLSTM}(x_t, \vec{h}_{t-1}, \overleftarrow{h}_{t+1}) \quad (1)$$

$$\alpha_{t,s} = \text{softmax}\left(\frac{Q_t \cdot K_s^T}{\sqrt{d_k}}\right) \quad (2)$$

$$\text{context}_t = \sum_s \alpha_{t,s} \cdot V_s \quad (3)$$

$$\hat{y}_{t+1:t+72} = \text{Linear}(\text{context}_T) \quad (4)$$

Architecture parameters:

- Hidden size: 128, Layers: 2, Dropout: 0.3
- Attention heads: 4, Key dimension: 32
- Optimizer: AdamW, Scheduler: Cosine warm restarts
- Mixed precision training (FP16) enabled

5.4 Hyperparameter Optimization

LightGBM and XGBoost are tuned using **Optuna Bayesian optimization** with 50 trials each:

Listing 3: Optuna HPO for LightGBM

```

1 def objective(trial):
2     params = {
3         "num_leaves": trial.suggest_int("num_leaves", 20, 150),
4         "learning_rate": trial.suggest_float("lr", 0.01, 0.3, log
5             =True),
6         "n_estimators": trial.suggest_int("n_estimators", 100,
7             1000),
8         "min_child_samples": trial.suggest_int("min_child", 5,
9             50),
10        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
11        "colsample_bytree": trial.suggest_float("colsample", 0.6,
12            1.0),
13        "reg_alpha": trial.suggest_float("alpha", 1e-8, 10.0, log
14            =True),
15        "reg_lambda": trial.suggest_float("lambda", 1e-8, 10.0,
16            log=True),
17    }
18    # 5-fold TimeSeriesSplit cross-validation
19    cv_scores = cross_val_score(model, X, y, cv=tscv, scoring="
20        neg_rmse")
21    return -cv_scores.mean()

```

5.5 Evaluation Metrics

All 9 models are evaluated using a temporal train/test split (80/20) on 34,824 data points. The table below reflects the latest production training run:

Table 8: Model Performance (Latest Training Run — 27,859 train / 6,965 test samples)

Model	RMSE	MAE	R ²	MAPE (%)
Ridge (Champion)	1.55	0.87	0.9988	0.64
Gradient Boosting	1.69	0.87	0.9986	0.58
Extra Trees	2.05	1.00	0.9979	0.67
XGBoost (Optuna)	2.25	1.19	0.9975	0.77
LightGBM (Optuna)	2.27	1.19	0.9975	0.76
Random Forest	4.33	2.39	0.9908	1.51
SVR	6.14	3.25	0.9815	2.04
ElasticNet	18.73	14.97	0.8282	9.94
Bi-LSTM + Attention	28.94	21.19	0.5913	14.56

Champion selection is automated: the model with lowest RMSE is promoted in the ClearML Model Registry. All models are versioned regardless of rank.

6 Explainability

6.1 SHAP (SHapley Additive exPlanations)

Two SHAP explainer types are implemented:

1. **TreeExplainer:** For tree-based models (LightGBM, XGBoost, Random Forest). Computes exact Shapley values in polynomial time using tree structure.
2. **GradientExplainer:** For the PyTorch Bi-LSTM model. Computes expected gradients as SHAP value approximations.

The SHAP explainer provides:

- Per-sample feature contributions (top- k features)
- Global feature importance (mean $|\text{SHAP}|$)
- Direction indicators (increase/decrease/neutral)
- Serialization for deployment (`.pk1` format)

6.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME generates local surrogate explanations for any black-box model:

Listing 4: LIME explanation example

```
1 from training_pipeline.explainability import LIMEExplainer
2
3 lime_exp = LIMEExplainer(
4     model=trained_model,
5     training_data=X_train,
6     feature_names=feature_names,
7     mode="regression",
8     kernel_width=0.75,
9 )
10
11 # Explain a single prediction
12 result = lime_exp.explain_instance(X_test[0], num_samples=5000)
13 # Returns: predicted_value, local_r2, contributions list
```

LIME capabilities:

- Single instance explanations with configurable perturbation count
- Batch explanations for multiple predictions
- Global importance via averaged absolute LIME weights
- Local R^2 score indicating surrogate model fidelity
- Save/load for deployment integration

6.3 Temporal Grad-CAM

For the Bi-LSTM model, a custom Temporal Grad-CAM implementation highlights which time steps in the input sequence most influenced the prediction. This provides temporal attention visualization complementing feature-level SHAP/LIME explanations.

7 CI/CD, Testing, and Automation

7.1 GitHub Actions Workflows

Six modular workflows automate the full ML lifecycle from data collection through deployment verification:

Table 9: Automated CI/CD Pipelines

Workflow	Trigger	Actions
ci.yml	PR / Push to main	Ruff linting, pytest unit tests, import smoke tests
feature-pipeline.yml	Hourly cron	Ingest real-time data from AQICN and OpenWeather APIs, apply 37-feature engineering, push to ClearML Feature Store
train.yml	Daily 2AM UTC	Extract training data, train all 9 models, compute SHAP/LIME, register all models, upload artifacts
deploy-api.yml	Push to backend paths	Pre-deploy validation (endpoint sweep, <code>render.yaml</code> syntax check), trigger Render deploy hook, post-deploy health and endpoint verification
deploy-frontend.yml	Push to web_app/	Pre-deploy build check (TypeScript, ESLint, Next.js build), wait for Vercel auto-deploy, run HTTP smoke tests
integration.yml	Every 6 hours	Live API contract validation against the production Render endpoint (health, models, predict, simulate, shadow)

7.2 GitHub Secrets and Variables

All credentials are stored as encrypted GitHub Actions secrets. Non-sensitive configuration is stored as repository variables:

Table 10: Repository Secrets and Variables

Name	Type	Purpose
AQICN_API_KEY	Secret	AQICN data platform token
OPENWEATHER_API_KEY	Secret	OpenWeatherMap API key
CLEARML_API_ACCESS_KEY	Secret	ClearML access key
CLEARML_API_SECRET_KEY	Secret	ClearML secret key
RENDER_DEPLOY_HOOK_URL	Secret	Render deploy webhook URL
API_URL	Variable	https://pearls-aqi-api.onrender.com
FRONTEND_URL	Variable	Vercel deployment URL
PYTHON_VERSION	Variable	3.11

7.3 Test Suite

The project includes a pytest suite covering all pipeline stages:

Table 11: Test Modules and Coverage

Module	Coverage Scope
<code>test_api.py</code>	31 Flask endpoint tests: health, predict, models, explain, simulate, historical, satellite, shadow, error handling
<code>test_data_pipeline.py</code>	14 tests: synthetic data generation, feature engineering, temporal encodings, wind decomposition, lag features
<code>test_e2e_pipeline.py</code>	8 end-to-end integration tests: ingest → transform → train → registry → predict → drift → anomaly
<code>test_feature_pipeline.py</code>	ClearML Dataset creation, feature store retrieval
<code>test_training_pipeline.py</code>	Model training completion, metrics are finite, evaluation pipeline
<code>load_test.py</code>	Locust load test configuration (50 concurrent users, weighted endpoints)

Listing 5: Running the test suite

```

1 # Full test suite with verbose output
2 pytest tests/ -v
3
4 # With coverage report
5 pytest tests/ --cov=. --cov-report=term-missing --cov-report=html
6
7 # Run specific module
8 pytest tests/test_api.py -v --tb=short

```

All tests use mocked external dependencies (API calls, ClearML connections) to enable fast, deterministic execution without network access.

8 Deployment and Infrastructure

8.1 Production Architecture

The production system uses a split deployment strategy optimized for the free tier of modern cloud platforms:

Table 12: Production Deployment Architecture

Component	Platform	Details
Frontend (Next.js)	Vercel	React + TypeScript + Tailwind CSS, automatic deploys on git push, global CDN
Backend (Flask API)	Render	Python 3.11, Gunicorn with 2 workers, automatic deploys from GitHub, free tier with auto-sleep
Feature Store	ClearML	Dataset versioning with Parquet artifacts, experiment tracking, model registry
CI/CD	GitHub Actions	6 modular workflows: CI, feature pipeline, training, deploy-api, deploy-frontend, integration tests

8.2 Backend Deployment (Render)

The Flask REST API is deployed on Render’s free web service tier and is publicly accessible at:

Live Backend API: <https://pearls-aqi-api.onrender.com>

Listing 6: Render start command

```

1 gunicorn deployment.api.main:app --bind 0.0.0.0:$PORT --workers 2
2   \
  --timeout 120 --max-requests 1000 --max-requests-jitter 50

```

- **Runtime:** Python 3.11, auto-detected from `requirements-deploy.txt`
- **Build:** `pip install -r requirements-deploy.txt` (lightweight subset excluding PyTorch/CUDA)
- **Health Check:** GET `/health` endpoint (automatic monitoring)
- **Environment:** API keys injected via Render’s encrypted environment variables
- **Cold Start:** Free tier sleeps after 15 minutes of inactivity; first request takes ~30s
- **Deploy Hook:** A webhook URL triggers redeployment from the `deploy-api.yml` GitHub Action

8.3 Frontend Deployment (Vercel)

The Next.js dashboard is deployed on Vercel with the following configuration and is publicly accessible at:

Live Frontend Application: <https://aqi-predictor-3cawg37a4-giki.vercel.app>

- **Framework:** Next.js (App Router) with React
- **Root Directory:** deployment/web_app
- **Build Command:** npm run build
- **Environment Variable:** NEXT_PUBLIC_API_URL pointing to the Render backend
- **10 React Components:** ParticleWindEngine, ModelZooSelector, AtmosphericBentoGrid, VignetteAlert, ActualVsPredictedGraph, CausalPolicySimulator, EdgeInferenceEngine, SatelliteParticleMap, ShadowCanaryMonitor, LiquidNavigation
- **ErrorBoundary:** React class component wrapping all pages for graceful error recovery
- **12 Test Files:** Vitest + React Testing Library covering all components

8.4 Frontend Component Architecture

The Next.js dashboard is structured around 10 purpose-built React components, each handling a distinct piece of the interface:

Table 13: Frontend React Components

Component	Responsibility
ParticleWindEngine	Animated particle background that adjusts density and color based on the current AQI reading
ModelZooSelector	Dropdown allowing users to switch between all 9 registered models for inference
AtmosphericBentoGrid	72-hour bento-grid forecast with color-coded AQI levels per hour
VignetteAlert	Full-screen red vignette overlay triggered when AQI exceeds the hazardous threshold
ActualVsPredictedGraph	Plotly chart comparing model predictions against observed values
CausalPolicySimulator	Counterfactual slider interface for simulating traffic reduction and crop-burning scenarios
EdgeInferenceEngine	Client-side ONNX WebAssembly inference toggle for offline predictions
SatelliteParticleMap	Simulated Sentinel-5P NO ₂ column grid visualization
ShadowCanaryMonitor	Champion vs. challenger shadow-traffic comparison dashboard
LiquidNavigation	Animated navigation bar with smooth page transitions

8.5 Docker Configuration

For local development and self-hosted deployment, a multi-stage Dockerfile optimizes for minimal image size:

Listing 7: Multi-stage Dockerfile architecture

```

1 # Stage 1: Builder (compiles native dependencies)
2 FROM python:3.11-slim AS builder
3 RUN apt-get install -y gcc g++ cmake
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt
6
7 # Stage 2: Runtime (production-optimized)
8 FROM python:3.11-slim AS runtime
9 RUN groupadd -r appuser && useradd -r -g appuser appuser
10 COPY --from=builder /opt/venv /opt/venv
11 COPY . /app
12 USER appuser
13 EXPOSE 8000
14 CMD ["gunicorn", "deployment.api.main:app", "--bind", "
    0.0.0.0:8000",
15     "--workers", "2", "--timeout", "120"]

```

Security measures include: non-root execution, no build tools in runtime image, no secrets baked into layers, and an explicit health check probe.

8.6 API Reference

The Flask backend exposes 9 endpoints, each available at both root path and /v1/ prefix. Authentication via X-API-Key header is optional (enabled when API_AUTH_KEY env var is set). Rate limiting: 60 requests/minute per IP.

Table 14: REST API Endpoints

Method	Endpoint	Description
GET	/health	Liveness and readiness check (uptime, model status)
POST	/predict	72-hour AQI forecast with RMSE-based confidence intervals
POST	/explain	SHAP feature contributions
POST/GET	/explain/lime	LIME local surrogate explanations
GET	/historical	Historical AQI data (paginated)
GET	/models	List all registered models with metrics
POST	/simulate	Counterfactual policy simulation
GET	/satellite/sentinel5p	Simulated Sentinel-5P grid data
GET	/shadow/metrics	Shadow model comparison metrics

8.7 Local Development Setup

Listing 8: Complete local development workflow

```

1 # 1. Clone and create virtual environment
2 git clone https://github.com/Zain-ul-abdeen-773/AQI-Predictor.git

```

```
3 cd AQI-Predictor
4 python -m venv venv && source venv/bin/activate # Linux/Mac
5 pip install -r requirements.txt
6
7 # 2. Configure environment variables
8 cp .env.example .env
9 # Edit .env with: AQICN_API_KEY, OPENWEATHER_API_KEY, CLEARML
   keys
10
11 # 3. Generate training data (fetches real historical data from
   Open-Meteo)
12 python -m data_pipeline.backfill --start-date 2022-07-28
13
14 # 4. Run exploratory data analysis
15 python -m eda.run_eda && python -m eda.advanced_eda
16
17 # 5. Train all 9 models
18 python -m training_pipeline.train
19
20 # 6. Launch Flask API backend
21 flask --app deployment.api.main:app run --port 8000
22
23 # 7. Launch Next.js frontend (separate terminal)
24 cd deployment/web_app && npm install && npm run dev
25
26 # 8. Run tests
27 pytest tests/ -v --cov=. --cov-report=term-missing
28
29 # 9. Docker deployment (alternative to steps 6-7)
30 docker-compose up --build
```

9 Conclusion and Future Work

9.1 Project Summary

The Pearls AQI Predictor delivers a complete, production-grade MLOps solution for air quality forecasting in Sargodha, Pakistan. The system addresses a critical public health need — with 36.7% of observed hours classified as hazardous, accurate multi-day forecasting enables preventive action for vulnerable populations.

Table 15: Project Deliverables Summary

Component	Deliverable
Data Pipeline	Real-time ingestion (AQICN + OpenWeather) + historical back-fill (Open-Meteo), 43,801+ hourly observations
Feature Engineering	37 engineered features with leakage prevention, cyclical encoding, lag features, rolling statistics
Model Development	9 trained models spanning 4 paradigms with Optuna Bayesian HPO (50 trials), all versioned in ClearML
Explainability	SHAP (TreeExplainer + GradientExplainer) + LIME (TabularExplainer) + Temporal Grad-CAM
EDA	19-stage analysis producing 20 visualizations and 26 statistical reports
Automation	6 GitHub Actions workflows: CI, hourly ingestion, daily training, deploy-api, deploy-frontend, live integration tests
Frontend	Next.js 16 dashboard with 11 interactive React components, ErrorBoundary, model zoo selector, causal simulator, and particle engine
Deployment	Backend on Render + Frontend on Vercel + Docker for local/self-hosted use
Testing	pytest suite (70+ tests across 6 modules) + 12 frontend Vitest tests + Locust load tests
Documentation	LaTeX technical report + OpenAPI 3.1 spec + CONTRIBUTING.md + README with architecture diagrams

9.2 Key Achievements

- Achieved $R^2 = 0.9988$ (Ridge champion) on 72-hour ahead AQI forecasting with MAPE of 0.64% and RMSE of 1.55
- Trained and versioned **all 9 models** — users can select any model for inference through the frontend model zoo
- Built a fully automated pipeline requiring **zero manual intervention** for ongoing production operation
- Demonstrated **Granger causality** linking temperature, wind speed, and pressure to AQI — validating the scientific basis for using meteorological features
- Identified **thermal inversions** (low wind + cool temperatures) as the primary driver of hazardous events in Sargodha through comprehensive EDA
- Implemented **three complementary explainability methods** providing both global and local model interpretability at feature and temporal levels
- Deployed a **production-ready system** with an interactive Next.js dashboard, REST API with 9 endpoints (+ /v1/ versioning), API authentication, rate limiting, request tracing, Sentry error monitoring, and 6 automated CI/CD workflows

- Achieved **100% frontend component test coverage** with 12 Vitest test files and 70+ backend pytest tests including end-to-end pipeline integration tests

9.3 Limitations

- Training data availability limited by Open-Meteo API coverage (historical data from 2022 onwards)
- LightGBM/XGBoost Optuna HPO requires sufficient training samples; small datasets may cause convergence issues
- Free-tier deployment (Render) introduces cold-start latency (~ 30 s after 15 minutes of inactivity)
- Model assumes stationarity of pollution sources; structural changes require retraining
- Current confidence intervals use model RMSE scaled by $\sqrt{1 + h/12}$ with proper z-scores (1.28 for 80%, 1.96 for 95%); formal conformal prediction would provide distribution-free theoretical guarantees
- The Bi-LSTM + Attention model underperforms classical models on this dataset ($R^2 = 0.59$), likely due to the relatively small training set size for a deep learning approach

9.4 Future Work

1. **Multi-City Expansion:** Extend to Lahore, Karachi, Islamabad, and Faisalabad with city-specific fine-tuned models
2. **Satellite Integration:** Incorporate MODIS Aerosol Optical Depth (AOD) and Sentinel-5P NO₂ column data for spatial pollution mapping
3. **Ensemble Stacking:** Combine top-3 models via a meta-learner for improved predictions beyond any individual model
4. **Online Learning:** Implement incremental updates using river/Vowpal Wabbit to adapt without full retraining
5. **Mobile Alerts:** Firebase Cloud Messaging push notifications for hazardous AQI predictions targeting sensitive groups
6. **Causal Discovery:** Apply PC algorithm or NOTEARS for causal graph structure learning between pollution sources
7. **Conformal Prediction:** Replace heuristic confidence intervals with distribution-free coverage guarantees
8. **Model Monitoring:** Integrate Evidently AI for automated data drift and model degradation detection in production

A Environment Variables

Table 16: Complete Environment Variable Reference

Variable	Default	Description
AQICN_API_KEY	demo	AQICN API token
OPENWEATHER_API_KEY	–	OpenWeatherMap key
CLEARML_API_ACCESS_KEY	–	ClearML access key
CLEARML_API_SECRET_KEY	–	ClearML secret key
TARGET_CITY	Sargodha	Forecast target
FORECAST_HORIZON_HOURS	72	Prediction window
LOOKBACK_WINDOW_HOURS	72	Input sequence length
AQI_ALERT_THRESHOLD	150	Alert trigger level
LSTM_HIDDEN_SIZE	128	LSTM hidden dim
LSTM_NUM_LAYERS	2	LSTM layers
OPTUNA_N_TRIALS	50	HPO budget
LOG_LEVEL	INFO	Logging verbosity
API_AUTH_KEY	–	API authentication key (empty = disabled)
SENTRY_DSN	–	Sentry error tracking DSN
CORS_ORIGINS	Vercel domains	Comma-separated allowed origins
SERVER_HOST	0.0.0.0	API bind host
SERVER_PORT	8000	API bind port

B Project File Structure

Listing 9: Complete project structure

```

1 .
2 +-- config/           Configuration (Pydantic BaseSettings)
3 +-- eda/              Exploratory Data Analysis (19 analyses)
4 +-- data_pipeline/    Data ingestion & feature engineering
5 +-- feature_pipeline/ ClearML Feature Store management
6 +-- training_pipeline/ Model training, evaluation,
   explainability
7 |   +-- models/       9 model implementations
8 |   |   +-- baseline.py Ridge / ElasticNet
9 |   |   +-- ensemble_trees.py GradientBoosting / RF / ExtraTrees / SVR
10 |   |   +-- tree_ensemble.py LightGBM + Optuna
11 |   |   +-- xgboost_model.py XGBoost + Optuna
12 |   |   +-- deep_learning.py Bi-LSTM + Multi-Head Attention
13 |   |   +-- callbacks.py EarlyStopping, Checkpoints
14 |   |   +-- grad_cam.py Temporal Grad-CAM
15 |   +-- registry.py   Multi-model versioning + champion
   promotion
16 +-- deployment/

```

17	+-- api/ versioning)	Flask REST API (9 endpoints + /v1/
18	+-- web_app/ components)	Next.js 16 dashboard (11 React
19	+-- streamlit_app/	Streamlit dashboard (alternative UI)
20	+-- tests/	pytest suite (70+ tests, 6 modules)
21	+-- docs/	This documentation
22	+-- .github/workflows/	CI/CD (6 modular workflows)
23	+-- openapi.yaml	OpenAPI 3.1 specification
24	+-- CONTRIBUTING.md	Developer setup guide
25	+-- Dockerfile	Multi-stage build
26	+-- docker-compose.yml	Container orchestration
27	+-- render.yaml	Render deployment config
28	+-- vercel.json	Vercel frontend config
29	+-- requirements.txt	Full Python dependencies
30	+-- requirements-deploy.txt	Lightweight deploy dependencies